

NOM PRÉNOM :

Les Q1 à Q6 ont été enlevée volontairement

Q7 - Instruction pour l'importation du module gpxpy

```
import gpxpy 100 from gpxpy import *
```

10,25

Q8 - Valeur numérique renvoyée par `mystere(I1)`. Signification de cette valeur

```
777 1050 10,25
```

Cette valeur correspond à l'altitude moyenne de iti 10,25

10,5

Q9 - Complexité temporelle de `mystere` en fonction de la taille n de la liste passée en argument

Complexité temporelle de `mystere` est $O(n)$ car on parcourt la liste une seule fois.

10,5

Q10 - Fonction `altitude_maximale(iti:itineraire) -> float`

```
def altitude_maximale(iti):  
    res = iti[0][2]  
    for (lat, long, alt) in iti:  
        if res < alt:  
            res = alt  
    return res
```

0,25 si pas géré 3-uplet
0,5 si utilise max
0,75 si trouve

1

Q11 - Fonction `denivele_global(iti:itineraire) -> float`

```
def denivele_global(iti):  
    return altitude_maximale(iti) - iti[0][2]
```

10,5

Q12 - Fonction deniveler_positif_cumule(iti:itineraire) -> float

```
def deniveler_positif_cumule(iti):
    s = 0
    for i in range(len(iti) - 1):
        diff = iti[i+1][2] - iti[i][2]
        if diff > 0:
            s += diff
    return s
```

si passage par liste 1,25
si par bon range 1

1,5

Q13 - Fonction alt_glissante(liste_alt:list, p:int) -> list

```
def alt_glissante(liste_alt, p):
    res = []
    for i in range(len(liste_alt)):
        j = min(p - 1, len(liste_alt) - i - 1)
        s = 0
        for k in range(i, i + j + 1):
            s += liste_alt[k]
        res.append(s / (j + 1))
    return res
```

1/2

Q14 - Complexité de la fonction alt_glissante(liste_alt:list, p:int) -> list

$O(n \times p)$ On a une boucle $O(n)$
qui contient une boucle pour
en $O(p)$.

1/1

OAS 8 pas de justification



Numéro d'inscription

Numéro de table

Né(e) le

Nom :

Prénom :

Emplacement
QR Code

Filière : PC

Session : 2025

Épreuve de : INFORMATIQUE

Consignes

- Remplir soigneusement l'en-tête de chaque feuille avant de commencer à composer
- Rédiger avec un stylo non effaçable bleu ou noir
- Ne rien écrire dans les marges (gauche et droite)
- Numéroté chaque page (cadre en bas à droite)
- Placer les feuilles A3 ouvertes, dans le même sens et dans l'ordre

PC5IN

Q15 - Type des variables lat_ref et long_ref. Explication du contenu de ces variables dans le contexte de ce sujet

Ce sont des listes (type list).
Ces listes contiennent des float correspondant
aux latitudes (lat_ref) et longitudes (long_ref)
de tous les points du globe de la ligne

10,5

Q16 - Signature des fonctions auxiliaire et principal. Justifier

- auxiliaire (x: list, y: list) → list
On compare x et y à des listes vides.
Return x, y ou z. z est une liste car on append.
- principale (x: list) → list
x est une liste, on effectue des slices.
renvoie le résultat d'une fonction qui renvoie
une liste.

1

Q17 - Type(s) de programmation utilisé(s) pour coder la fonction principal

- | | |
|---|---|
| ☐ Itératif | ☐ Algorithme glouton |
| ☒ Récursif | ☐ Programmation dynamique |
| ☐ k plus proches voisins | ☒ Diviser pour régner |

10,5

NE RIEN ÉCRIRE DANS CE CADRE

Q18 - Justification de la terminaison de la fonction principal

Auxiliaire termine car si se ou y est vide la fonction termine directement et sinon soit se soit y diminue de taille (soit se est pas soit y).

Principal termine car si se de taille inférieure à 1, la fonction s'arrête et sinon les appels à principal sont sur des listes de taille inférieure et auxiliaire termine.

Pour principal termine pour toutes listes.

1

Q19 - Nom décrivant le type de tri utilisé. Justification de votre choix

Le tri utilisé est le tri fusion.

On divise la liste en 2 listes de taille $n/2$. On tri récursivement les 2 listes et on fusionne les 2 listes triées.

1

Q20 - Compléter les lignes 5, 6, 8, 10, 12, 15 et 17 de la fonction ref

1,75

Ligne 5 :

0

Ligne 6 :

$\text{len}(\text{liste} - \text{ref}) - 1$

Ligne 8 :

$(\text{ind} - \text{deb} + \text{ind} - \text{fin}) // 2$

Ligne 10 :

k

Ligne 12 :

$\text{ind} - \text{deb} = k + 1$

Ligne 15 :

$\text{liste} - \text{ref}[\text{ind} - \text{fin}]$

Ligne 17 :

$\text{liste} - \text{ref}[\text{ind} - \text{deb}]$

Q21 - Fonction standardise(liste_parcours:itineraire) -> itineraire

```
def standardise ( liste_parcours ) :  
    res = []  
    for p in liste_parcours :  
        lat = ref ( p[0], lat_ref )  
        long = ref ( p[1], long_ref )  
        alt = dem E ( lat, long )  
        res.append ( ( p[0], p[1], alt ) )  
    return res
```

12

Partie III

Q22 - Explication des lignes 15 à 19 de la fonction mystere2. Nom du type d'algorithme utilisé

On cherche le successeur de strophe le plus proche pas encore visité.
Comme on fait des choix locaux jusqu'à une solution il s'agit d'un algorithme glouton

11

Q23 - Résultat de l'instruction mystere2(G, "a", "f")

```
>>> ([ 'a', 'c', 'd', 'e', 'f' ], 8)
```

11

Q24 - Ce programme permet-il au randonneur de trouver le chemin de difficulté cumulée minimale ? Vous justifierez votre réponse.

Non ce programme n'est pas optimal.
On peut faire 6 en faisant $a \rightarrow d \rightarrow e \rightarrow f$

Q25 - Tableau ci-dessous à compléter

Étape	sTraite	distance						aVisiter
		a	b	c	d	e	f	
Initialisation		0, a	X	X	X	X	X	["a"]
1	a	0, a	3, a	2, a	4, a	X	X	["b", "c", "d"]
2	c	0, a	3, a	2, a	4, a	X	X	["b", "d"]
3	b	0, a	3, a	2, a	4, a	8, b	X	["d", "e"]
4	d	0, a	3, a	2, a	4, a	5, d	8, d	["e", "f"]
5	e	0, a	3, a	2, a	4, a	5, d	6, e	["f"]

Q26 - Compléter les lignes 8, 9, 10 et 15 du code

Ligne 8 :

$s! = sLimit$

Ligne 9 :

$s = distance[s][1]$

Ligne 10 :

$chemin.append(s)$

Ligne 15 :

$distance[sFin][0]$

Q27 - Proposition pour diminuer le nombre de tests d'appartenance à une liste de l'algorithme de Dijkstra

On pourrait utiliser des ensembles au lieu de listes pour aVisiter et degaVisiter.
On gagnerait en complexité en moyenne : Recherche en $O(1)$ contre $O(n)$ pour une liste de taille n .



Numéro d'inscription

Numéro de table

Né(e) le

Nom :

Prénom :

Emplacement
QR Code

Filière : PC

Session : 2025

Épreuve de : INFORMATIQUE

Consignes

- Remplir soigneusement l'en-tête de chaque feuille avant de commencer à composer
- Rédiger avec un stylo non effaçable bleu ou noir
- Ne rien écrire dans les marges (gauche et droite)
- Numéroté chaque page (cadre en bas à droite)
- Placer les feuilles A3 ouvertes, dans le même sens et dans l'ordre

PC5IN

Q28 - Liste des sommets visités :

- par l'appel dijkstra(G1, "a1", "j1")

["a1", "b1", "c1", "d1", "e1", "f1", "g1", "h1", "i1", "j1"]

- par l'appel dijkstra(G1, "j1", "a1")

["j1", "g1", "c1", "a1"]

Q29 - Dernière ligne de chacun des tableaux ci-dessous à compléter.

Étape	Traité	distanceF			distanceB		
		a	b	c	d	e	f
Init.		0, a ∞ , f	∞ , a ∞ , f	∞ , a ∞ , f	∞ , a ∞ , f	∞ , a ∞ , f	∞ , a 0, f
1	a, F	0, a ∞ , f	3, a ∞ , f	2, a ∞ , f	4, a ∞ , f	∞ , a ∞ , f	∞ , a 0, f
2	f, B	0, a ∞ , f	3, a ∞ , f	2, a ∞ , f	4, a 4, f	∞ , a 1, f	∞ , a 0, f
3	e, B	0, a ∞ , f	3, a 6, e	2, a ∞ , f	4, a 2, e	∞ , a 1, f	∞ , a 0, f
4	d, B	0, a 6, d	3, a 4, d	2, a 6, d	4, a 2, e	∞ , a 3, f	∞ , a 0, f

Étape	Fmin	Bmin	BFmin	aVisiterF	aVisiterB
Init.	0	0	∞	["a"]	["f"]
1	2	0	∞	["b", "c", "d"]	["f"]
2	2	1	8	["b", "c", "d"]	["d", "e"]
3	2	2	6	["b", "c", "d"]	["d", "b"]
4	2	4	6	["b", "c", "d"]	["b", "a", "c"]

NE RIEN ÉCRIRE DANS CE CADRE

Q30 - Justification de la proposition pour trouver le chemin de longueur minimale

Non par exemple ici le sommet D est atteint en premier par Backward et Forward et la somme est de 8. Alors que l'optimal est E.

1/1

Q31 - Compléter les lignes 4, 5, 13, 21, 22 et 23 de la fonction dijkstra_bidirectionnel

Ligne 4 :

[s]

Ligne 5 :

[]

Ligne 13 :

$B_{\min} > F_{\min} + B_{\min}$

Ligne 21 :

distance F

Ligne 22 :

distance B

Ligne 23 :

distance B[v][c] + distance F[v][c]

Q32 - Démonstration que s_i appartient à `dejaVisitesF` ou `dejaVisitesB`

Par l'absurde. Supposons qu'il existe un sommet s_i qui n'appartient ni à `dejaVisitesF` ni à `dejaVisitesB`.
Donc la distance depuis le sommet de départ est $\geq F_{\min}$ et $\geq B_{\min}$.

La longueur totale du chemin passant par s_i est donc supérieure ou égale à $F_{\min} + B_{\min}$.

Par la condition d'arrêt on aurait cette distance supérieure ou égale à $B_{\min}$ ce qui contredit l'hypothèse.

Tout sommet s_i du chemin appartient obligatoirement à `dejaVisitesF` ou `dejaVisitesB`.

1/1

Q33 - Existence de i_0 tel que s_{i_0} a été visité à une étape forward et s_{i_0+1} à une étape backward

$S = s_0, s_1, \dots, s_n$

Donc s_0 est visité par forward et s_n visité par backward. Par la Q32 on sait que chaque sommet de S est soit déjà visité F ou déjà visité B, il y aura donc obligatoirement un moment où l'on passe d'un ensemble à l'autre. Donc il existe tel que $s_{i_0} \in \text{déjà visité F}$ et $s_{i_0+1} \in \text{déjà visité B}$

Q34 - Correction partielle de l'algorithme de Dijkstra bidirectionnel

Comme $s_{i_0} \in \text{déjà visité F}$, $\text{distance F}[s_{i_0}][c] = d(s, s_{i_0})$
et $s_{i_0+1} \in \text{déjà visité B}$, $\text{distance B}[s_{i_0+1}][c] = d(s_{i_0+1}, t)$

$\text{distance F}[s_{i_0+1}][c] \leq \text{distance F}[s_{i_0}][c] + \text{poids}(s_{i_0}, s_{i_0+1})$
car on visite les voisins de s_{i_0} s'il est déjà visité F.

La longueur du chemin est donc $d = d(s, s_{i_0}) + \text{poids}(s_{i_0}, s_{i_0+1}) + d(s_{i_0+1}, t)$

donc $d = \text{distance F}[s_{i_0}][c] + \text{poids}(s_{i_0}, s_{i_0+1}) + \text{distance B}[s_{i_0+1}][c]$

On en déduit que $d \geq \text{distance F}[s_{i_0+1}][c] + \text{distance B}[s_{i_0+1}][c]$

Comme s_{i_0} est déjà visité F et s_{i_0+1} déjà visité B.

on a $\text{BFmin} \leq \text{distance F}[s_{i_0+1}][c] + \text{distance B}[s_{i_0+1}][c]$

Donc $d \geq \text{BFmin}$, ce qui contredit notre hypothèse qui stipulait que $d \leq \text{BFmin}$. Par

l'algorithme de Dijkstra Bidirectionnel trouve bien la distance minimale entre s et t .

FIN

